

# Mini Report

Team 10, Week 8

## Participating Team Members

Imogen: Wrote tests to increase *generate\_report* line coverage to 100%.  
Jennie: Wrote new tests for *add\_purchases* and *calculate\_shipping\_fee\_for\_heavy\_items* to improve code coverage  
Amrutha: Refactored the *add\_purchases()* and *add\_customer()* methods, updating test cases where necessary.  
Folademilade & Emmanuel: Refactored *calculate\_shipping\_fee*, *calculate\_shipping\_fee\_for\_heavy\_items* and *calculate\_shipping\_fee\_for\_fragile\_items* into one method and added new tests where necessary.  
Anastasiia: Refactored *generate\_report()*

## What refactoring techniques did you apply?

To refactor the *add\_purchases()* method, the inline function method was used.  
To refactor *calculate\_shipping\_fee*, *calculate\_shipping\_fee\_for\_heavy\_items* and *calculate\_shipping\_fee\_for\_fragile\_items*, the parametrisation function technique was used.

## What was the reason for applying these techniques?

To eliminate the redundancy of *calculate\_shipping\_fee\_for\_heavy\_items* and *calculate\_shipping\_fee\_for\_fragile\_items* and making program easier to understand

## How do you know that observable behaviour has not changed after refactoring?

By updating the regression test suite to 100% line coverage, then, upon a refactor change, running this suite and observing that they all pass.

## What are the (potential) benefits and drawbacks of applying these techniques?

It makes the code more readable while preserving the behaviour.  
Tests may need to be changed due to updated method constructors

## Overall, how do you feel about the refactoring process? Worth it? Why (or why not)?

If done incrementally before the piece of code is deployed, it could be a very worthwhile process, as most of the time the refactoring done is either minor or basic but brings a lot of value to the final code, maintainability and readability-wise.

## 1. Tests added to achieve 100% line coverage

Python

```
def test_add_purchases(self):
    cm = CustomerManager()
    name = "Alice"
    purchases = [{'price': 50, 'item': 'banana'}, {'price': 20, 'item':
'apple'}]
    cm.add_purchases(name, purchases)

    self.assertEqual({name: purchases}, cm.customers)
```

Python

```
def test_heavy_item_shipping_fee(self):
    purchases = [{'price': 70, 'weight': 25}]
    purchases_under_threshold = [{'price': 70, 'weight': 15}]

    fee = calculate_shipping_fee_for_heavy_items(purchases)
    self.assertEqual(fee, 50)

    under_threshold_fee =
calculate_shipping_fee_for_heavy_items(purchases_under_threshold)
    self.assertEqual(under_threshold_fee, 20)
```

Python

```
# generate_report() unit tests:
# There are many different branches to cover, separated into different
# tests.
# Furthermore, the function doesn't return anything or mutate any external
# variables. The printed values must be asserted against instead, via StringIO.
```

```
def test_generate_report_discount_eligible_vip_customer(self):
    cm = CustomerManager()
    purchases = [
        {'price': 1001} # VIP customer price, and above Discount threshold
    ]
    cm.add_customer("Bob", purchases)
    printed = io.StringIO()
    sys.stdout = printed
    cm.generate_report()
    self.assertEqual(
        printed.getvalue(),
        "Bob\nEligible for discount\nVIP Customer!\n"
    )
```

```

def test_generate_report_discount_eligible_priority_customer(self):
    cm = CustomerManager()
    purchases = [
        {'price': 801} # Priority customer price, above Discount threshold
    ]
    cm.add_customer("Bob", purchases)
    printed = io.StringIO()
    sys.stdout = printed
    cm.generate_report()
    self.assertEqual(
        printed.getvalue(),
        "Bob\nEligible for discount\nPriority Customer\n"
    )

def test_generate_report_discount_potential(self):
    cm = CustomerManager()
    purchases = [
        {'price': 301} # Below Discount thres, but potential (>300 <500).
    ]
    cm.add_customer("Bob", purchases)
    printed = io.StringIO()
    sys.stdout = printed
    cm.generate_report()
    self.assertEqual(
        printed.getvalue(),
        "Bob\nPotential future discount customer\n"
    )

def test_generate_report_no_discount_or_potential_untaxed(self):
    cm = CustomerManager()
    purchases = [
        # Significantly below Discount threshold. And below tax threshold, so
        # won't be taxed either.
        {'price': 5}
    ]
    cm.add_customer("Bob", purchases)
    printed = io.StringIO()
    sys.stdout = printed
    cm.generate_report()
    self.assertEqual(printed.getvalue(), "Bob\nNo discount\n")

```

## 2. Refactoring

```
42
43     def calculate_shipping_fee(self, purchases):
44         heavy_item = False
45         for purchase in purchases:
46             if purchase.get('weight', 0) > 20:
47                 heavy_item = True
48                 break
49         if heavy_item:
50             return 50
51         else:
52             return 20
53
54     def calculate_shipping_fee_for_heavy_items(purchases):
55         for purchase in purchases:
56             if purchase.get('weight', 0) > 20:
57                 return 50
58         return 20
59
60     def calculate_shipping_fee_for_fragile_items(purchases):
61         fragile_item = False
62         for purchase in purchases:
63             if purchase.get('fragile', False):
64                 fragile_item = True
65                 break
66         if fragile_item:
67             return 60
68         else:
69             return 25
70
```

A screenshot of the original code before refactoring

*def calculate\_shipping\_fee\_for\_heavy\_items, def calculate\_shipping\_fee\_for\_fragile\_items and calculate\_shipping\_fee* were merged using the parametrise function refactoring technique with the extra input, *check\_for\_fragile\_items*. This was introduced because the method for calculating shipping fee when fragility was in focus defaulted the fee to 25 rather than 20, which was the default value in *calculate\_shipping\_fee* and *calculate\_shipping\_fee\_for\_heavy\_items*.

A drawback was that the test suite was built around manually testing each function, and now extra effort will be needed to also refactor the tests so that one function is tested under different cases.

Python

```
def calculate_shipping_fee(self, purchases, check_for_fragile_items=False):
    heavy_item = False
    fragile_item = False

    for purchase in purchases:
        if purchase.get('weight', 0) > 20 :
            heavy_item = True
        if purchase.get('fragile', False):
            fragile_item = True
    if heavy_item:
        return 50
    elif fragile_item:
        return 60
    else:
        return 25 if check_for_fragile_items else 20
```

New refactored function

Python

```
def test_shipping_fee_with_heavy_item(self):
    cm = CustomerManager()
    purchases = [{'price': 100, 'weight': 25}]

    fee = cm.calculate_shipping_fee(purchases)
    self.assertEqual(fee, 50)

def test_shipping_fee_with_fragile_item(self):
    cm = CustomerManager()

    purchases = [{'price': 70, 'fragile': True}]

    fee = cm.calculate_shipping_fee(purchases, check_for_fragile_items=True)
```

```

self.assertEqual(fee, 60)

def test_shipping_fee_with_checking_for_fragile_items(self):
    cm = CustomerManager()

    purchases = [{'price': 70, 'fragile': False}]

    fee = cm.calculate_shipping_fee(purchases, check_for_fragile_items=True)
    self.assertEqual(fee, 25)

def test_no_special_items_shipping_fee(self):
    cm = CustomerManager()
    purchases = [{'price': 40, 'weight': 5, 'fragile': False}]

    fee = cm.calculate_shipping_fee(purchases)
    self.assertEqual(fee, 20)

```

Inline function:

```

def add_purchase(self, name, purchase):
    self.add_customer(name, [purchase])

def add_purchases(self, name, purchases):
    self.add_customer(name, purchases)

```

After refactoring:

```

def add_purchases(self, name, purchases):
    self.add_customer(name, purchases)

```

Test updated:

|                                                                                                                                                                                                                                                         |                                                                                                                     |                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> def test_add_purchase(self):     cm = CustomerManager()     name = "Alice"     purchase = {'price': 50, 'item': 'banar'}     cm.add_purchase(name, purchase)      self.assertEqual(         {name: [purchase]},         cm.customers     ) </pre> | <div style="border: 1px solid #ccc; padding: 2px; width: 20px; margin: 0 auto;">             →<br/>+         </div> | <pre> def test_add_purchase(self):     cm = CustomerManager()     name = "Alice"     purchase = {'price': 50, 'item': 'banana'}     cm.add_purchases(name, [purchase])      self.assertEqual(         {name: [purchase]},         cm.customers     ) </pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Removed the if else block by using a defaultdict.

```
def add_customer(self, name, purchases):  
    if name in self.customers.keys():  
        self.customers[name].extend(purchases)  
    else:  
        self.customers[name] = purchases
```

After refactoring:

```
from collections import defaultdict  
class CustomerManager:  
    def __init__(self):  
        self.customers = defaultdict(list)  
        self.tax_rate = 0.2  
        self.tax_threshold = 100  
        self.discount_threshold = 500  
  
    def add_customer(self, name, purchases):  
        self.customers[name].extend(purchases)
```

generate\_report() refactoring:

Before:

```
def generate_report(self):
    for y, x in self.customers.items():
        a = 0

        for z in x:
            if z['price'] > self.tax_threshold:
                taxed_price = z['price'] * (1 + self.tax_rate)
                a += taxed_price
            else:
                a += z['price']
        print(y)

    if a > self.discount_threshold:
        print("Eligible for discount")
    else:
        if a > 300:
            print("Potential future discount customer")
        else:
            print("No discount")
    if a > 1000:
        print("VIP Customer!")
    else:
        if a > 800:
            print("Priority Customer")
```

After:

```
def generate_report(self):
    for c, p in self.customers.items():
        total_price = 0

        for item in p:
            if item['price'] > self.tax_threshold:
                taxed_price = item['price'] * (1 + self.tax_rate)
                total_price += taxed_price
            else:
                total_price += item['price']

        print(c)

        self.customerBenefits(total_price)

    def customerBenefits(self, total_price):
        print(self.isDiscount(total_price))
        print(self.customerStatus(total_price))

    def isDiscount(self, total_price):
        if total_price > self.discount_threshold:
            return "Eligible for discount"
        else:
            return "Potential future discount customer" if total_price > self.FUTURE_DISCOUNT else "No discount"

    def isPriority(self, total_price):
        return "Priority Customer" if total_price > self.PRIORITY else ""

    def customerStatus(self, total_price):
        return "VIP Customer!" if total_price > self.VIP else self.isPriority(total_price)
```



When refactoring `generate_report()`, firstly, the variables were renamed to improve readability in the for loops. Secondly, the entire if statement to print whether the customer gets discounts and whether they are VIP, was replaced by extracting the method. Furthermore, the newly created method was extracted into 2 additional methods for the 2 respective customer attributes they describe, as stated above.

To improve readability even further, the nested if statements were refactored into inline if statements. To achieve this effect, it was necessary to extract an additional method that perhaps would have been better left as a parameterised method.